

EXPERIMENT NO. 11

Develop a Single Page Application (SPA) dashboard that polls a REST API

Name: Rohit Pujari

Roll No.: 28

Date: _____

Aim:

To develop “ServerPulse”, a single page monitoring dashboard served by Express on port 3011 that polls the REST endpoints `/api/servers` and `/api/servers/:id`, shows the CPU and memory usage of six simulated servers with health badges, opens a detail view with a canvas line chart of CPU history through hash routing, and makes polling robust with `AbortController` time-outs, exponential back-off on errors, a Pause button and automatic pausing when the browser tab is hidden.

Theory:

A Single Page Application loads one HTML page and then updates only parts of it with JavaScript, using JSON data fetched from a REST API. Moving between views does not reload the page. HTTP is a request-response protocol, so the server cannot push new values to the page by itself; the simplest way to keep the dashboard current is polling, i.e. asking the server again at regular intervals. (WebSockets and Server-Sent Events can push data, but need extra server support.)

REST endpoints and simulated data

Method and path	Response	Used by
GET <code>/api/servers</code>	{ time, servers: [{ id, name, region, cpu, mem, status }] }	Overview, every poll
GET <code>/api/servers/:id</code>	One server plus history (last 30 CPU readings); 404 for unknown id	Detail view
GET <code>/</code>	index.html, app.js and style.css via <code>express.static</code>	First page load

On the server, `setInterval()` produces a new reading every 2 seconds. CPU follows a random walk around a base value for each server, and `worker-01` is deliberately unreliable and is reported as down about half of the time. The status is calculated from the numbers: down, critical (CPU above 85% or memory above 90%), warning (CPU above 70%) or healthy.

The polling loop

- **Recursive `setTimeout()`:** the next poll is scheduled only after the previous response has been handled, so slow responses never overlap, and the delay can change after every poll. `setInterval()` would fire at a fixed rate regardless.
- **`AbortController`:** each request gets a signal; a 5-second timer aborts a request that hangs, and a new poll (for example after a view change) aborts the one still running.
- **Exponential back-off:** after a failure the delay doubles (3, 6, 12, 24, then at most 30 s) and a red banner shows the retry time; after the next success it returns to 3 s. This avoids flooding a server that is already in trouble.
- **Page Visibility API:** when `document.hidden` becomes true (tab in the background) the `visibilitychange` event stops polling; it restarts immediately when the tab becomes visible.
- **cache: 'no-store'** makes sure the browser never answers a poll from its HTTP cache.

Situation	Next poll
Successful response	after 3 s
Network error, HTTP error or 5 s timeout	previous delay x 2, maximum 30 s
Pause button pressed	none until Resume
Tab hidden	none until the tab is visible again
Hash changed (another view)	immediately

Hash routing and rendering

The views are chosen from `location.hash: "#/server/5".split('/')` gives `["#", "server", "5"]`, so the router knows which endpoint to call. A `hashchange` event starts a new poll at once. The part after `#` is never sent to the server, so no extra server routes are needed, the Back button works and each view can be bookmarked. Cards are built with template literals, `reduce()` counts servers per status for the summary row, and the width of each usage bar is animated with a CSS transition. The detail view draws the CPU history on a `<canvas>`

with the 2D context: grid lines at 0, 50 and 100%, then `moveTo()`/`lineTo()` for each reading, scaled to the height of the canvas.

Important Code:

app.js - router and polling with back-off

```
function route() {
  const [, page, id] = location.hash.split('/');
  return page === 'server' ? { name: 'detail', id: Number(id) } : { name: 'overview' };
}

async function poll() {
  clearTimeout(timer);
  controller?.abort(); // cancel a request that is still
running
  controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(), 5000);
  const r = route();
  const url = r.name === 'detail' ? `/api/servers/${r.id}` : '/api/servers';
  try {
    const res = await fetch(url, { signal: controller.signal, cache: 'no-store' });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const data = await res.json();
    r.name === 'detail' ? renderDetail(data) : renderOverview(data.servers);
    lastOk = Date.now(); delay = BASE_DELAY; banner.hidden = true;
  } catch (err) {
    if (err.name === 'AbortError' && !paused) return; // superseded by a newer poll
    delay = Math.min(delay * 2, MAX_DELAY);
    banner.textContent = `Cannot reach server (${err.message}). Retrying in ${delay / 1000}
s...`;
    banner.hidden = false;
  } finally {
    clearTimeout(timeout);
  }
  schedule();
}

function schedule() {
  if (!paused && !document.hidden) timer = setTimeout(poll, delay);
}
```

app.js - pause when the tab is hidden

```
document.addEventListener('visibilitychange', () => { // stop polling in a background tab
  if (document.hidden) clearTimeout(timer); else if (!paused) poll();
});
```

server.js - REST endpoints

```
app.get('/api/servers', (req, res) =>
  res.json({ time: new Date().toISOString(), servers: servers.map(summary) }));
app.get('/api/servers/:id', (req, res) => {
  const s = servers.find(x => x.id === Number(req.params.id));
  if (!s) return res.status(404).json({ error: 'No such server' });
  res.json({ ...summary(s), history: s.history });
});
```

Output:

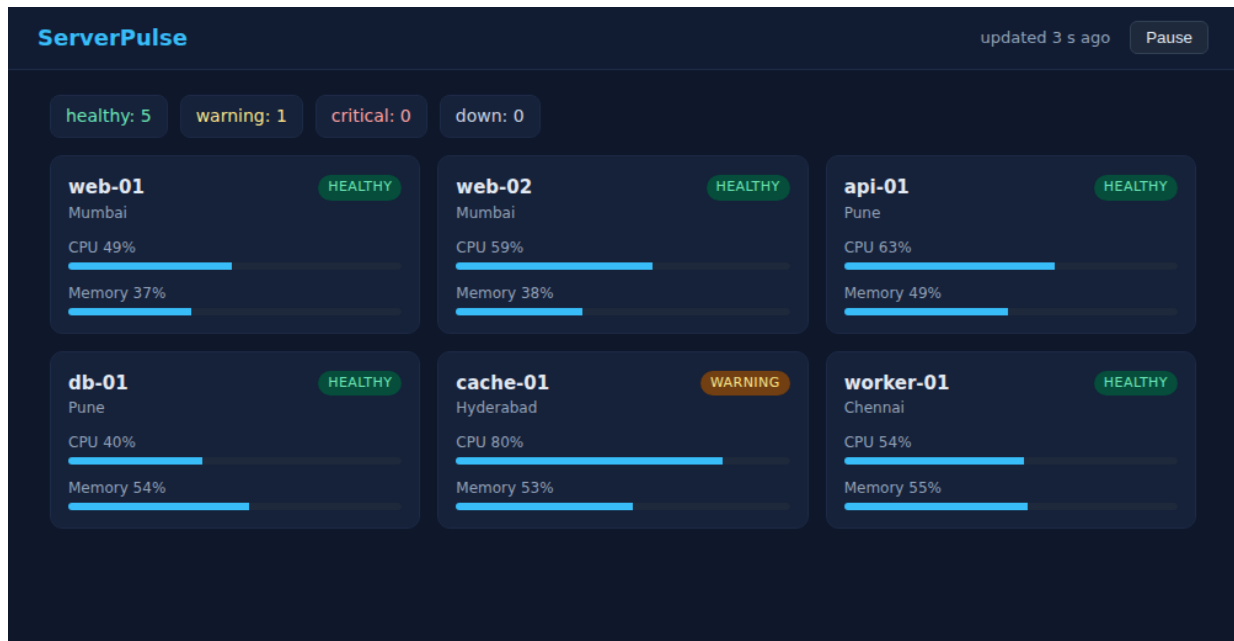


Figure 11.1: Overview view (#/) - live CPU and memory of six servers with the status summary

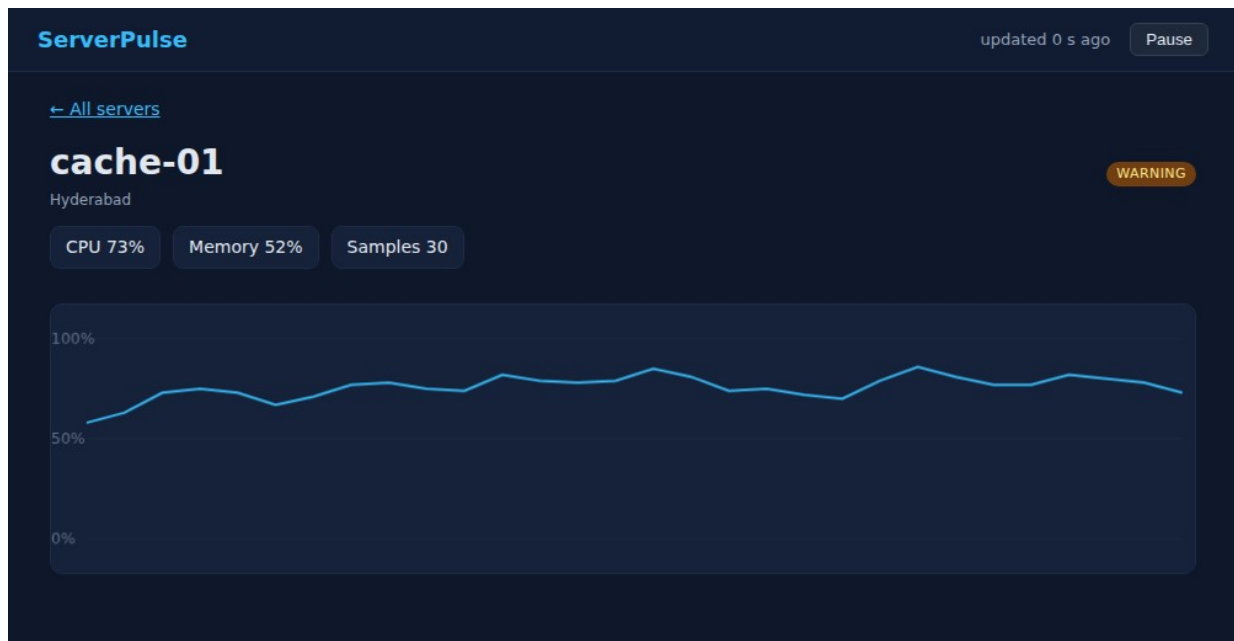


Figure 11.2: Detail view (#/server/5) - CPU history of cache-01 drawn on a canvas

Conclusion:

A single page monitoring dashboard was developed that polls an Express REST API. Hash routing switched between the overview and detail views without page reloads, and the canvas chart showed the CPU history. Recursive `setTimeout()`, `AbortController` time-outs, exponential back-off and the Page Visibility API made the polling efficient and robust.